


**COMPREHENSIVE USER GUIDE**

# CorpusMind

Local-first, AI-native research environment
for corpus linguistics and multimodal discourse analysis

| ENGINE | PLATFORMS | LICENSE |
|---|---|---|
| Python + FastAPI + spaCy | Win / Mac / Linux / PWA | AGPL-3.0 |

**v1.2.14-3**

Dr. Waleed Mandour
Sultan Qaboos University | ORCID: 0000-0002-9262-5993

Prof. Wesam Ibrahim
Princess Nourah Bint Abdulrahman University | ORCID: 0000-0003-0710-6038

PART 1

# Citation

If you use CorpusMind in your research, please cite it as:

> *Mandour, W., & Ibrahim, W. (2026). *CorpusMind: A local-first, AI-native research environment for corpus linguistics and multimodal discourse analysis* (Version 1.2.14-3) [Computer software]. Zenodo. https://doi.org/10.5281/zenodo.21226650*

PART 2

# 1. What is CorpusMind?

CorpusMind is a research tool for corpus linguists and discourse analysts. It lets you upload texts, run concordance searches, compute collocations, keyness, dispersion, and vocabulary profiles, analyse grammar and discourse features under named academic taxonomies, research learner language, and work with Arabic corpora using CAMeL Tools. A Vision Suite (and the companion **CorpusMind Lens** app) extends the same workflow to images using Kress and van Leeuwen's Visual Grammar and eight discourse lenses.

The AI Assistant answers questions about your corpus by calling the analysis tools and citing specific evidence (concordance line IDs, computed statistics). Every answer is either **grounded** (backed by a tool call) or clearly flagged as **ungrounded** — never silently presented as fact. A floating version of the same assistant is available from every analysis screen.

Everything runs on your own machine. No data leaves your computer unless you explicitly choose to use a cloud AI provider or the optional Gemini error-interpretation feature.

PART 3

# 2. Installation

## Option A: Desktop App (recommended)

Download the installer for your platform from the releases page: Windows (EXE/MSI), macOS Apple Silicon and Intel (DMG), Linux (AppImage/deb). Install Ollama as well — the app starts it automatically and downloads recommended models with one click from **Settings → Model Providers**.

## Windows SmartScreen note

CorpusMind is a free, independently published academic tool, and its installers are not code-signed with a paid certificate. The first time you launch the Windows installer, Microsoft Defender SmartScreen may therefore show **"Windows protected your PC"** and stop the setup. This is expected for unsigned installers — it is a reputation warning, not a virus detection. Windows users should simply accept the note: click **More info**, then **Run anyway**, and the installation will continue normally. If your browser flags the `.msi` file as "not commonly downloaded", choose **Keep** / **Keep anyway** rather than discarding it, and if you want to double-check the download first, compare its SHA-256 checksum against `SHA256SUMS.txt` attached to the same release.

The desktop app automatically:

- Starts the Python engine in the background (and waits until it is healthy)
- Starts Ollama in the background (if installed)
- Restarts either backend if it goes down while the app is running (v1.2.6 self-heal)
- On macOS, closing the window no longer stops the backends (v1.2.5)

## Option B: Development Mode

You need **Python 3.12**, **Node.js 20**, and **Ollama**.

```
ollama pull llama3.2:3b
```

Set up the engine:

```
cd CorpusMind/engine python3 -m venv .venv source .venv/bin/activate pip
install -e ".[dev]" python -m spacy download en_core_web_sm corpusmind-engine
```

Set up the web app (new terminal):

```
cd CorpusMind/web npm install npm run dev
```

Open http://localhost:5173. The engine listens on `127.0.0.1:8765` (override with `CORPUSMIND_PORT` / `CORPUSMIND_HOST`).

## Optional: Arabic Support

Arabic morphology analysis (roots, patterns, dialect identification) needs the CAMeL Tools data pack, about 168 MB. There are three ways to get it:

1. **From inside the app (recommended)**: Settings has an **Arabic data pack** card; the Arabic Tools error card offers the same Install button. The installer downloads the pinned packages from CAMeL Lab's official releases, verifies the size and SHA256 of every download, and shows progress; you can cancel at any time.
2. **From a terminal** (headless machines):

```
pip install camel-tools pyrsistent muddler cachetools emoji future regex
camel_data -i morphology-db-msa-r13 camel_data -i dialectid-model6
```

1. **Bundled at build time** (maintainers only): a desktop build made with `CORPUSMIND_BUNDLE_CAMEL_DATA=1` ships the pack inside the app, so Arabic Tools work offline with no install step.

If the data is missing, the Arabic Tools panel shows a clear error with an Install button instead of silently hanging; the engine never downloads data at request time. Only the explicit installer downloads anything.

Note: the morphology data (`calima-msa-r13`) is GPL-2.0-only licensed data, which is why the default desktop build does not redistribute it; see THIRD_PARTY_LICENSES.md for the full statement of what is and is not redistributed.

---

PART 4

# 3. Creating a Project and Uploading Texts

1. Click **Corpora Selection → Your Corpus** in the sidebar.
2. Select an existing project from the dropdown, or click **+ New Project**.
3. Click **+ New** under Corpora to create a corpus (name, language, genre).
4. Click your corpus to select it.
5. Click the upload area (or drag and drop) to add text files.

Supported formats: TXT, DOCX, PDF, HTML, XML, CSV, Markdown. The engine detects encoding, cleans the text, tokenizes, tags POS, lemmatizes, and parses dependencies. For Arabic corpora it uses CAMeL Tools (if installed) for morphology, root extraction, and dialect identification. The pipeline recipe (model names and versions) is visible and exportable.

## Corpus Cleaning

Each corpus has a **Clean** button with 16 options: collapse whitespace, remove URLs/emails, lowercase, remove punctuation/numbers/emoji, remove stopwords, minimum token length, and Arabic-specific normalization (alef variants, diacritics, tatweel). Cleaning is destructive — it re-processes every document.

## Reference Corpus

**Corpora Selection → Reference Corpus** offers three options: **Download** (HuggingFace + Wikipedia + OPUS), **Upload** your own, or the **bundled** BE06 frequency list.

---

PART 5

# 4. Concordance Search (KWIC)

1. Select a corpus and open **Concordance**.

2. Type a query (`*` works as a wildcard); choose word, lemma, or POS level.
3. Set the context window and click **Search**.

Each line has a stable ID (`doc:0:3`) that the AI Assistant can cite as evidence. Results export to five formats.

## CQL mode (corpus query language, v1.2.13)

Switch the concordancer from **Simple** to **CQL** to search for whole *patterns* instead of single nodes. The syntax is **CQP-flavoured but not CQP** — quoted values are wildcard patterns (not regexes), regex only via `/re/` and unanchored, scoping is `within sentence | document` only. What you can type:

- Literal words: `"risk"` — wildcards allowed: `"book*"`, `"coll??ate"`.
- Token attributes: `[lemma="take"]`, `[pos="NOUN"]`, `[word="book" & pos="NOUN"]` (`&` = and, `|` = or; `!=` for negation; `/…/` for regular expressions: `[word=/colou?r/]`).
- Attributes available: `word`, `lemma`, `pos` (UPOS), `xpos`, `rel` (UD dependency), `morph` (features), and — for Arabic — `root` and `pattern`, which few corpus tools expose at all. The **real stored format** (verified against the CAMeL calima-msa-r13 backend) is a DOTTED root and a template pattern with digit radical slots: `[root="▫.▫.▫"]` matches / / , and `[pattern="1▫2▫3"]` matches -type nouns.
- Sequences and gaps: `[lemma="take"] []{0,3} "risk"` (0–3 any-token gap), quantifiers `?`, `+`, `{m,n}` and star (zero-or-more), groups with alternation `(cat|dog)`.
- Flags: `%c` ignore case (Unicode-aware folding — école/École, STRASSE/ straße, final sigma, Cyrillic), `%d` fold Arabic diacritics — inside or after a token: `[lemma="kitab" %c]`. CQL is case-sensitive by default (CQP convention); the Simple box is not.
- Scoping: append `within sentence` (alias `within s`) or `within document` to constrain every match; by default a sequence may cross a sentence boundary.

Results render in the same KWIC table with sort, seeded sampling, pagination, and **server-side export** (the engine re-runs the query, so the file contains the full match set, not just the on-screen page). An invalid pattern returns the position of the error, e.g. *"Invalid CQL query: syntax error at position 12: …"*, and the concordancer draws a caret under the offending character. Patterns run in a worker thread under an explicit work budget and a wall-clock deadline: an over-expensive pattern (e.g. an unbounded gap with no literal anchor) returns an actionable message — *"Narrow it: add a literal anchor, bound unbounded gaps ([]\ → []{0,50}), or scope with 'within sentence'"'* — instead of freezing the app, and a running query can be cancelled while in flight. Students get the same CQL surface under a stricter budget. One match per start is reported (the earliest-ending span). The 20,000-token candidate cap per query remains; the result panel flags when the total is a lower bound.

### CQL statistics (v1.2.13-2)

The CQL match node feeds the statistics tools as well: **frequency**, **collocations**, **dispersion**, and **n-grams** all accept a CQL pattern (`cql_query`), so you can sketch, say, the collocates of `[pos="ADJ"] "fox"` or the dispersion of `[root="▫.▫.▫"]` across documents with the same association measures as the simple endpoints. Subcorpora can be created **from a CQL query** (documents containing at least one match become the member set), queries can be saved per project, and the grounded AI assistant gains a `search_cql` tool (tool-capable models only).

### Vector KWIC (Semantic Search)

**Vector KWIC** finds lines by *meaning*, not just form: pick an embedding model (`bge-m3`, multilingual, or `nomic-embed-text`, English), download it with one click, and search with natural-language queries (e.g. *"sentences about economic hardship"*). Ranking is raw cosine similarity over a local SQLite vector cache — no external API. On CPU-only machines the first search embeds the corpus in small batches and can take a while; see Troubleshooting §13.4.

---

PART 6

# 5. Frequency, Collocation, Keyness, and Dispersion

**Frequency**: word/lemma/POS lists with per-million and percent; STTR at the top.

**Collocation**: node word + window size; all 7 measures (MI, T-score, log-likelihood, Dice, LogDice, chi-square, Delta P) plus the interactive collocation network graph. A "Collocates" slider (6–40) in the network toolbar sets how many collocates to show per node — it grows BOTH levels, the node word's collocates and each expanded collocate's own, and applies on release; node sizes rescale with graph size and the view re-fits so the network always stays inside its window (exportable as SVG/PNG/JSON).

**Keyness**: target vs reference corpus; significance (log-likelihood, chi-square) beside effect size (Log Ratio, %DIFF, Simple Maths, Odds Ratio). **Methods PDF** auto-drafts a methodology paragraph for your manuscript.

**Dispersion**: Juilland's D and Gries' DP across documents.

---

PART 7

# 6. Advanced Analysis Tools

**N-grams**: n = 2–10 with Biber et al.-style frequency-and-range criteria for lexical bundles.

**POS Analysis**: distributions and POS n-grams; switch tagsets (UPOS, Penn Treebank, CLAWS7, CAMeL for Arabic) from Settings.

**Grammar**: passive voice, modals, negation, relative clauses, complex NPs, and tense from the dependency parse.

**Dependency**: query any UD relation (nsubj, obj, obl, ...) for common governor–dependent pairs.

**Discourse**: a user-selectable, citable taxonomy of discursive features:

- **Hyland (2005) metadiscourse** — interactive (transitions, frame markers, endophorics, evidentials, code glosses) and interactional (hedges, boosters, attitude markers, self-mentions, engagement) categories.
- **Halliday & Hasan (1976) cohesion** — reference pronouns, the four conjunction classes, and lexical repetition chains across adjacent sentences (substitution/ellipsis are not covered).
- **Martin & White (2005) Appraisal** — engagement (entertain, attribute, deny, counter, proclaim), graduation-force intensifiers, and an inscribed affect starter set (invoked attitude is not covered).
- **CLAWS/USAS semantic tagset (top-level)** — the bundled USAS lexicon maps every word onto one of 24 semantic categories (communication, cognition, emotion, politics...), each annotated with its discourse-functional group. Lexicon-based lookup — where the lexicon has no entry, tokens are honestly reported as *unmatched*; this is not the licensed CLAWS/USAS tagger. Availability is shown in Settings so the lexicon can be confirmed active before an analysis.
- **SFG Transitivity & Modality (Halliday & Matthiessen 2014)** — process types (material, mental, relational, behavioural, verbal, existential) and modality values, derived from the stored Universal Dependencies parse.
- **Persuasion Index (Gong et al. 2026)** — 15 interpretable dimensions of persuasive language per document, grouped by the classical appeals (Logos, Ethos, Pathos) and shown as a radar chart. The package is version-pinned (persuasion-index 0.3.0, Apache-2.0) and ships inside the app. It measures rhetorical *strategies*, not whether arguments are true. The lens reports which optional linguistic resources are active versus degraded to neutral baselines.
- **Cialdini 2007 — Influence, Revised Edition, ch. 2–7 (new in v1.2.10)** — the six persuasion principles (reciprocation, commitment & consistency, social proof, liking, authority, scarcity) as lexical cue sets, with a profile radar chart. Cue hits are **indicator counts, not mechanism classifications**: "expert" in a sentence is evidence of an authority *cue*, not proof an authority *appeal* was deployed. Cue-list precision is unvalidated pending gold-sample review (the words "official", "certified" and "leading" are known precision risks), so read counts as exploratory. The 2021 "Unity" principle is not covered. Each pair of categories also reports **co-occurrence** — the number of sentences containing cues from both; co-occurrence is not causation.
- **Comparing two corpora (keyness)** — pick a comparison corpus and every category row gains the standard keyness battery: log-likelihood, Log Ratio (Hardie 2014), %DIFF and the simple-maths heuristic, with a low-power warning when counts are small. The comparison

visuals follow one palette everywhere: the target corpus is always **green**, the reference corpus **purple**, and divergence charts read green = more frequent in the target, red = more frequent in the reference. The grouped chart plots both corpora side by side on the shared per-million scale; the diverging chart plots Log Ratio around a zero axis, with each category label on the opposite side of its bar's direction so text and bar never overlap.

**Cue-lens reading aids (v1.2.10)**. Every cue-based taxonomy now also ships a profile radar (each axis normalized to the most frequent category — a shape, not absolute rates), a co-occurrence table, and a language-coverage badge declaring which languages the lens's cue sets actually support (all four cue lenses are English-only; the USAS lexicon covers English and Arabic). The badges come from the engine's registry, not from the interface, so what the badge says is what the lens covers.

**Enabling the optional Persuasion Index resources (v1.2.9)**. Four optional resources refine specific subfeatures but are license-restricted (their providers forbid redistribution), so the app cannot ship them: Brysbaert single-word concreteness (.xlsx), multiword concreteness ratings (.csv, OSF), NRC-VAD v2.1 unigrams (non-commercial use; manual download), and a licensed LIWC dictionary (legacy .dic — LIWC-22 .dicx is not supported). Each unavailable resource in the Persuasion Index panel shows a "How to enable" guide with the official download link and the exact target path. The folder is `~/.corpusmind/pi-resources` on macOS/Linux and `C:\Users\<you>\.corpusmind\pi-resources` on Windows; it survives app updates. Quickest route: run `python scripts/install_pi_resources.py --all-open` for the two concreteness files, download NRC-VAD yourself from saifmohammad.com and pass the zip with `--nrc-vad-zip`, and copy your licensed LIWC `.dic` with `--liwc-file`. Restart the engine and the panel marks them . Power users can instead set `CORPUSMIND_PI_CONCRETENESS_FILE`, `CORPUSMIND_PI_MWE_CONCRETENESS_FILE`, `CORPUSMIND_PI_LIWC_FILE`, or `CORPUSMIND_PI_NRC_VAD_FILE` to explicit paths.

Every result names and cites its taxonomy, so findings are reportable and comparable across studies.

**Vocabulary**: frequency bands (K1, K2–K9, AWL, Off-list), rare and academic words.

**Sentiment**: layered, Appraisal-grounded analysis. One layer profiles the text with Appraisal cues (Attitude: Affect, Judgment, Appreciation; Engagement; Graduation; Martin & White 2005). Another scores every sentence for valence (−1 to +1) and emotion with a lemma-level lexicon, and a timeline shows the flow. The grammar layer reads the stored dependency parse: negation flips the polarity of what it scopes over ("not good" scores negative) and intensifiers strengthen it ("very good" scores stronger than "good"). The emotion layer ships with a bundled starter lexicon and reports its coverage honestly; for full 8-emotion coverage (Plutchik's categories) in English and Arabic, convert the official NRC Emotion Lexicon once with scripts/build_sentiment_lexicons.py and set the engine's CORPUSMIND_SENTIMENT_LEXICON_DIR to that folder (the Sentiment view shows the expected path). The app cannot ship the NRC data itself: its terms of use forbid redistribution.

**Metaphor**: verb-based metaphor *candidates*; LLM triage and human verification required before counting any as confirmed.

PART 8

# 7. Learner Research Suite

Purpose-built for learner-corpus studies (Granger 1998; Housen & Kuiken 2009):

- **CAF Report**: complexity, accuracy, and fluency indicators for the active corpus, including HD-D (heterogeneous hapax-based diversity).
- **CIA Compare**: compare two corpora (e.g. learner vs expert writing) across the CAF battery with effect sizes.
- **Error Patterns**: candidate learner-error patterns surfaced from POS and dependency signals, ranked for manual inspection — candidates, not verdicts.
- **AI-vs-learner comparator**: inspect how AI-generated text differs from learner text on the same indicators (research ethics: disclose AI use; see the Assistant's audit trail).

PART 9

# 8. Arabic Analysis

**Arabic Tools** in the sidebar:

- **Morphology**: root (al-jizr), pattern (al-wazn), lemma, POS, stem, Buckwalter transliteration, number, gender, broken plurals (CAMeL calima-msa-r13).
- **Roots**: all words sharing a triliteral root (k.t.b → kitab, maktaba, katib, yaktub).
- **Dialect ID**: MSA, Egyptian, Gulf, or Levantine (CAMeL DIDModel6).
- **Buckwalter / Dediacritize / Normalize**: script utilities.
- **Register**: Classical / MSA / Dialectal detection.
- **Translate**: Arabic–English lookup equivalents.
- **Export** (v1.2.13): every tool result can be saved as xlsx / csv / tsv / txt / json — the morphology, roots, and clitics tools export their on-screen table columns; Buckwalter, dediacritized, and normalized export an original/result pair; dialect and register export a probability table. CSV/TSV files carry a UTF-8 BOM so Arabic opens cleanly in Excel/Sheets.

Arabic normalization is also available inside corpus cleaning, and Arabic is supported end-to-end in Vector KWIC via the multilingual bge-m3 model.

PART 10

# 8b. Urdu, Hindi, and Farsi Corpora (v1.2.11)

Urdu (ur), Hindi (hi) and Farsi/Persian (fa) are supported as corpus languages alongside English and Arabic. Each language has its own normalizer, and the engine never applies Arabic rules to Persian or Urdu:

- **What works out of the box**: upload (TXT/DOCX/PDF/HTML), tokenization, sentence splitting (Hindi danda and ; Urdu full stop ), concordance (incl. wildcards and regex), frequency + STTR, collocations, keyness, dispersion, n-grams, Vector KWIC (bge-m3), Learner CAF diversity and complexity indices, and per-language stopword lists (editable in Settings → Word lists; built-in lists ship for all five languages).
- **Normalization** (the "Normalize spelling variants" option in the API, `normalize: true`): Persian and Urdu unify Arabic-keyboard lookalikes ( → , → ; Urdu also → ); Hindi folds nukta ( → ) and chandrabindu. ZWNJ (U+200C) inside Persian/Urdu words is preserved by default; the request-level `zwnj` mode offers keep/space/strip for matching.
- **POS, lemmatization, dependency parse**: available when the optional Stanza backend is installed on the machine running the engine (`pip install -e ".[urdu-hindi-farsi]"`, models download on first use). The packaged desktop app ships the tokenizer-only fallback; the pipeline recipe on your corpus always names the backend that produced the tags.
- **Honest limits**: sentiment, discourse lenses (beyond the bilingual USAS), vocabulary bands (K1/AWL), and learner error rules are English/Arabic resources and return an explicit "not available for this language" message instead of wrong results. Readability reports the language-neutral LIX/RIX scores (Flesch stays English-only).
- **Reference data**: top-1000 frequency baselines for all three languages ship bundled (derived from wordfreq 3.1.1, CC BY-SA 4.0 data) for keyness.

---

PART 11

# 8c. Recommended Models for the AI Assistant (v1.2.11)

Settings → Model Providers lists curated local models. Gemma 4 (Google, Apache-2.0) is recommended for grounded answers in the new languages:

- `gemma4:e4b` (about 6-7 GB with the recommended QAT or Q4_K_M quantization) is the best balance for laptops; it supports tool calling, so the assistant's grounded answers work.
- `gemma4:e2b` (about 4-5 GB) runs on smaller machines.
- `gemma4:12b` (about 7-8 GB quantized) for workstation quality.

- All sizes handle text and images; E2B/E4B have 128K context, 12B has 256K. Pull the model inside the app (Settings → Model Providers) — it is never bundled with CorpusMind. If Ollama reports the model needs a newer version, update Ollama from ollama.com first; the app tells you when that is the case instead of showing a generic "model missing" error.

PART 12

# 9. Vision Suite and CorpusMind Lens (Multimodal Analysis)

Images are treated as corpus documents with metadata, provenance, and query tools, analysed against explicit frameworks. The same suite lives in the main app and in the dedicated **CorpusMind Lens** app.

## 9.1 Building an image corpus

Create a project and corpus, then in **Your Corpus** (Lens) or the **Vision Suite** (main app) create an **image set**, document provenance (source, period, selection criteria), and drag images in (PNG, JPEG, WebP, GIF, TIFF, BMP; ≤ 25 MB each, 50 per batch). Upload extracts OCR text, colour and composition features, and EXIF/XMP metadata — GPS coordinates are deliberately never extracted.

## 9.2 Metadata and per-image analysis

Each image carries IPTC-Core-aligned fields (source, date, licence, genre, language, notes); **Tag All Images** applies fields set-wide. Per image:

- **Analyse** — colour, composition (information value, salience, balance, vectors), OCR.
- **Visual Grammar** — Kress & van Leeuwen (2006) metafunctions, each claim a scored hypothesis.
- **Align** — image-region ↔ text-span alignment with confidence.
- **Discourse lenses** — 8 frameworks: Social Semiotic, CDA (Fairclough, van Dijk, Wodak, Machin & Mayr), Persuasion, Framing, Narrative, Visual Metaphor, Emotion, Cultural. Provenance badges show heuristic vs vision-LM mode.
- **Facial analysis** (opt-in, off by default) — descriptive cues only, never identity recognition.

## 9.3 Set-level tools

Vision-LM descriptions with a local vision model, recurring-theme aggregation, OCR corpus tools (KWIC over OCR text, frequency lists, set-vs- set keyness by log-likelihood), spreadsheet export with provenance, and **Export OCR corpus** — the set's text as a `<doc>`-marked corpus file for the main text tools.

PART 13

# 10. The AI Assistant

**AI Assistant** in the sidebar opens a tool-using agent (not a chatbot): it selects and calls analysis tools, then answers from their output. Tool use **grounded** (green badge); otherwise **ungrounded** (orange). A **floating assistant** button (bottom-right) offers the same grounded chat from every analysis screen, with context about the view you are on.

**Confidence layer**: the Assistant self-assesses confidence; below 70% you answer 2–3 verification MCQs before the answer is revealed.

**Human verification**: Accept / Reject / Edit every answer; the decision is recorded in the audit trail and the AI-usage disclosure for your Methods section.

**Student mode** (Settings → Research & Reproducibility): hides the AI answer until the student writes their own interpretation.

Example questions: *"What are the top 10 content words?"*; *"Find 'however' in context"*; *"What are the strongest collocates of 'patient'?"*; *"What hedges does this author use?"*; *"Compare this corpus against the reference."*

PART 14

# 11. Export, Collaboration, and Privacy

**Export**: every analysis exports as Excel (`.xlsx`), CSV, TSV, plain text, or JSON via the Export dropdown. The Collocation network exports as SVG/PNG/JSON. **Methods PDF** (Keyness view) drafts the methodology paragraph with exact tools, versions, and formulas.

**Corpus Hub**: the Reference Corpus view searches open-access corpora — HuggingFace Datasets, live Wikipedia (CC-BY-SA), and 1,200+ OPUS parallel corpora. Downloads land in your browser; searches are proxied through the engine, so your own data never leaves the machine.

**Saved searches and bookmarks**: save queries with parameters; bookmark lines/statistics with notes.

**Project sharing**: mark a project shared (public/private) with a share token; sync events are audited.

**At-rest encryption**: optional AES-256-GCM for image files via `CORPUSMIND_ENCRYPTION_KEY` (the key is never stored on disk).

**Accessibility**: WCAG 2.1 AA target — focus indicators, skip link, high contrast mode, reduced motion, 44px touch targets, full RTL mirroring.

**Smart Troubleshooting**: backend errors appear in the taskbar. With a Gemini API key configured (Settings → Gemini Interpretation, off by default), errors are auto-interpreted in plain language with a likely cause and suggested fix; error context leaves the device only with your explicit

acknowledgment. **Report to developer** opens a pre-filled email.

PART 15

# 12. Web App versus Desktop Application

Both forms share the same UI, project format, and engine. The **web app** (hosted at https://corpus-mind-web.vercel.app/) runs in the browser — no installation, with the engine started by you or on a lab server. The **desktop application** (Tauri 2, Windows/ macOS/Linux) bundles UI + supervisor + engine sidecar: the supervisor spawns and health-waits the engine, auto-starts Ollama, self-heals backends, and writes `engine.stdout.log` / `engine.stderr.log` to the OS log directory.

| Capability | Web app | Desktop |
|---|---|---|
| Analytical tools & Vision Suite | Identical | Identical |
| Engine delivery | Manual start / lab server | Automatic (supervised, self-healing) |
| Ollama | Run locally yourself | Auto-spawned if on PATH |
| Offline | Browser page only; engine needed for analysis | Fully offline |
| File access | Browser dialog uploads | Native dialogs + data dir |
| Logs | Managed by you | Managed for you |
| Data residency | Local or lab server | Always local |
| Updates | Automatic on next visit | New installer |
| Best for | Trying it out, lab-server teams, locked-down machines | Sensitive corpora, classrooms, full offline |

PART 16

# 13. Troubleshooting Common Issues

Most issues are environmental (a backend that is down, a missing model, or another application interfering with local connections) rather than bugs. Work through this list before reporting a problem; the Smart Troubleshooting panel (taskbar → error → Details) may already name the cause.

## 13.1 Security or grammar software intercepting Ollama (HTTP 500)

**Symptom**: "HTTP 500 Internal Server Error" when chatting or running AI features, often on a fresh installation, even though Ollama is installed and the model exists.

**Known cause**: desktop applications that inspect or proxy local HTTP traffic can intercept CorpusMind's requests to Ollama (`127.0.0.1:11434`). Confirmed real-world case: **Grammarly** running in the background caused exactly this; quitting it made the errors stop. Antivirus "web shields", VPNs, and corporate proxies can do the same.

**Fix**: quit or temporarily disable the interfering application (e.g. right-click the Grammarly tray icon → Quit), or add CorpusMind and Ollama to its exclusions/allow-list, then retry. If your organisation manages the machine, ask IT to exempt loopback (`127.0.0.1`) traffic from inspection.

## 13.2 "Ollama is not running" (503)

The engine cannot reach Ollama on `127.0.0.1:11434`. Start Ollama (the desktop app tries to start it automatically), verify with `curl http://127.0.0.1:11434/api/tags`, and make sure at least one chat model is pulled (Settings → Model Providers offers one-click downloads). In browser mode you must start both the engine (`corpusmind-engine`) and Ollama yourself.

## 13.3 "Model not found — run ollama pull" (409)

The selected model is not installed. Download it in **Settings → Model Providers** (one click) or run `ollama pull <model>`. If the model *is* installed, update to the latest release — v1.2.2 fixed a tag-matching bug (`bge-m3` vs `bge-m3:latest`) that caused a false 409 after a successful download; no re-download is needed.

## 13.4 Vector KWIC is slow or times out (CPU-only machines)

Embedding a large corpus on CPU takes minutes of compute. Since v1.2.5 requests are chunked into small batches (override with `CORPUSMIND_EMBED_BATCH`), and since v1.2.4 the timeout is configurable (`CORPUSMIND_EMBED_TIMEOUT_S`). Warm the model first (Warm-up button), expect the first search to be the slowest (the vector cache persists), and prefer the smaller `nomic-embed-text` model for English-only corpora. If the engine reports `502 embedding_unreachable`, the Ollama connection dropped mid-request — check §13.1/§13.2 and retry; the desktop app will also offer to restart the backend (v1.2.6).

## 13.5 The engine is not reachable (web / manual mode)

Start it with `corpusmind-engine` and check `http://127.0.0.1:8765/api/v1/health`. If the port is busy, set `CORPUSMIND_PORT`. If the firewall prompted on first run, allow the engine. Desktop builds health-wait for up to 120 s at boot — on very slow disks or with aggressive antivirus, first launch can take that long.

## 13.6 macOS: window closed and now nothing responds

Since v1.2.5 closing the window keeps the engine and Ollama alive; click the dock icon to reopen. If a backend actually died, the app probes and restarts only what is down when you reopen (v1.2.6). If you are on an older version, closing the window stopped the backends — quit and relaunch instead.

## 13.7 Arabic tools are missing or fail

CAMeL Tools is optional. Install it (see §2) and download the two data files. If only some tools fail, check the engine log for the specific model (e.g. the dialect-ID model) and run the matching `camel_data -i` command.

### 13.8 Where are the logs? How do I report a problem?

Desktop logs live in the OS log directory as `engine.stdout.log` and `engine.stderr.log` (macOS: `~/Library/Logs/CorpusMind`; Windows: `%APPDATA%\CorpusMind\logs`; Linux: `~/.local/share/CorpusMind/logs` — names per platform convention). The taskbar error panel and **Settings → Diagnostics** (Run Diagnostics) summarise backend health. Use **Report to developer** in the error panel to open a pre-filled email to `w.abumandour@squ.edu.om`, or file an issue at https://github.com/waleedmandour/CorpusMind/issues with the log excerpt and your OS, app version, and Ollama model list.

---

PART 17

# 14. CorpusMind Compared with Other Corpus Tools

CorpusMind complements rather than replaces existing tools. Claims below are sourced to each tool's own documentation/publications (2025).

**AntConc** (Anthony, free, desktop) — excellent for teaching and quick KWIC on pre-processed corpora, but has no POS tagger, lemmatiser, or parser, one user-selected collocation statistic, and no Arabic morphology, AI assistant, or multimodal support. CorpusMind bundles the full annotation pipeline and the seven-measure collocation suite with provenance records.

**Sketch Engine** (Lexical Computing, subscription, cloud) — the reference implementation of Word Sketches with ready-made 100+ language corpora. Cloud processing makes it unsuitable for confidential corpora without a data-processing agreement. CorpusMind is local-first and free, but does not (yet) implement Word Sketches.

**#LancsBox** (Lancaster, free, desktop) — strong visualisation (Graph Coll) and BNC/BNC64 support; lighter on statistics and without Arabic morphology or AI assistance. CorpusMind implements the full seven-measure collocation and four-measure dispersion suites (Juilland's D, DP, ARF, AWT) with provenance.

**Voyant Tools** (Sinclair & Rockwell, free, web) — fast visual exploration for digital humanities; raw-token only, no standard collocation statistics or annotation. CorpusMind adds linguistic annotation, formal statistics, and reproducibility.

| Feature | CorpusMind | AntConc | Sketch Engine | #LancsBox | Voyant |
|---|---|---|---|---|---|
| Licence | AGPL-3.0 (free) | Freeware | Subscription | Free (academic) | Open source |
| Deployment | Desktop + Web (local-first) | Desktop | Cloud | Desktop | Web |
| Collocation measures | 7 | 1 | Multiple (Word Sketch) | Several | 1 (raw) |
| Keyness (LL + effect sizes) | Yes | LL, chi-square | Yes | Yes | No |
| Dispersion | D, DP, ARF, AWT | No | Yes | Visual | No |

| Feature | CorpusMind | AntConc | Sketch Engine | #LancsBox | Voyant |
|---|---|---|---|---|---|
| POS / lemma / dependency | Yes (bundled) | No | Yes | Limited | No |
| Arabic morphology | Yes (CAMeL) | No | Yes | No | No |
| Multimodal (images) | Yes | No | No | No | No |
| Grounded AI assistant | Yes (local LLM) | No | No | No | No |
| Provenance per operation | Yes (YAML) | No | Limited | No | No |
| Data residency | Local by default | Local | Cloud | Local | Cloud |

**Rule of thumb**: Voyant for a quick visual overview; AntConc/#LancsBox for teaching and quick searches; Sketch Engine for Word Sketches and ready-made corpora; CorpusMind for reproducible, framework-grounded, local-first analysis of text and images.

---

PART 18

# 15. Student Mode — Classroom Teaching with Student Devices (v1.2.9)

Student Mode turns your desktop app into a small classroom server: up to ~20 students on their own phones or tablets open a link/QR in their normal browser and get a **read-only** view of your corpora plus the analysis tools and the AI assistant. Students cannot upload, delete, recompile, change settings, manage models, or see your AI conversation history — the restriction is enforced in the engine itself, and the student UI simply does not offer those actions. Your own desktop experience is unchanged while the classroom runs.

**Starting a classroom (teacher side)**

1. Open **Settings → Student Mode — Classroom Server**.
2. Choose a connection mode:
   - **Secure (HTTPS, recommended)** — the app's built-in proxy generates a local certificate. Each student device performs a one-time trust step: open the certificate QR/link shown in Settings, install and trust the profile (iOS: Settings → Profile Downloaded; Android: install as a CA certificate), then open the classroom link. This step is real and intentional — school-managed Chromebooks/iPads may block profile installs, so test on one student device first.
   - **Simple (plain HTTP)** — no certificate and no trust step, but the traffic is unencrypted. Use it only on a closed, trusted classroom Wi-Fi. Students on such networks can see content in transit.
3. Press **Start classroom**. Share the **Student QR** (or the link plus the `cm_study_…` token) with the class. Use **New tokens** to rotate them between classes — old links stop working.

4. The card shows how many students are active (last 10 minutes) and an estimate of how many students your machine can serve with the selected **classroom model**. Classroom chats always run on that small model — never on the large model you may use for solo research. For smoother multi-student generation, restart Ollama with the `OLLAMA_NUM_PARALLEL` environment variable set (e.g. 4).

**If the app screen goes blank after you turn Student Mode on**, simply right-click anywhere in the app and choose **Refresh**. The desktop's embedded web view sometimes needs a single reload once the classroom server starts; nothing is lost — your corpora, settings, and the running classroom are unaffected.

**Joining a classroom (student side)**

Scan the teacher's QR or open the link. The first screen asks for the server address and access token (usually prefilled from the QR). After connecting, students see the analysis tools — concordance, frequency, collocation, keyness, dispersion, n-grams, POS, grammar, dependency, discourse, Vector KWIC, the learner suite, Arabic tools, and the AI assistant — over the teacher's corpora, with a clear "Student Mode" status and a Source Code link (the app is AGPL-3.0; students on a network are entitled to the source). To leave, use Exit Student Mode or open the link with `?mode=teacher`.

**Seat limit (protects your machine)**

The classroom enforces a hard cap on how many students can join at once, sized from your hardware: by default the limit is estimated automatically from your device's available memory (or GPU VRAM when present) and the size of the classroom model — the same probe behind the model-download guidance. When the classroom is full, new students see a friendly "classroom is full" message and can retry in a few minutes, while already-connected students keep working and you (the teacher) are never capped. The card shows the live seat usage ("Seats: 12 active of 18") and its source. To set a fixed number instead, type it into **Seat limit** and press **Apply**; press **Auto** to return to the device-sized estimate. The cap is also informed by `OLLAMA_NUM_PARALLEL`: more parallel slots let the model serve several students simultaneously, but each slot still costs memory.

**Audit log (anonymous)**

While Student Mode is on, the engine keeps an audit log of classroom activity — on by default, and toggleable in the card. It records, with every possible detail but no identities:

- each student joining (shown as an anonymous alias, `S-1`, `S-2`, …) and the number of students connected over the session;
- every analysis request (tool, route, status, duration);
- **each question a student asked the local LM and the full answer the model returned**, with the model name and response time;
- denials (routes students may not touch, classroom-full rejections).

The log is anonymous by design: students appear only as aliases; no names, device addresses, or tokens are ever written, and the mapping is kept in memory only, so the file cannot be de-anonymised later. Entries are stored locally as one JSON-lines file per day under `<data>/classroom/audit/` (rotated when a day file grows past 5 MB). Open the **Classroom**

**audit log** panel in the card to browse today's events with a summary (students joined, chats held), or open the folder to archive the raw files for your teaching records.

**Security notes.** The classroom server binds to your machine only; the bundled proxy stamps proxied requests and the engine refuses any classroom traffic without a valid teacher or student token. Anyone with the student token can read your corpora and run analyses — that is the point of a classroom — so share the QR only with your class and rotate tokens between groups. On untrusted networks, prefer Secure mode.

PART 19

# Getting Help

- GitHub: https://github.com/waleedmandour/CorpusMind/issues
- Live web app: https://corpus-mind-web.vercel.app/
- Build Guide: docs/BUILD_GUIDE.md
- Methodology Reference: docs/METHODOLOGY.md
- Zenodo DOI: https://doi.org/10.5281/zenodo.21226650